



WARWICK

Advanced Computer Graphics

Spatial Data Structures

Dr. Tom Bashford-Rogers

Motivation

- ▶ Raytracing against all triangles is SLOW!
- ▶ Can we do better?



Solutions

- ▶ Quicker Processors

- ▶ More cores

- ▶ Better Algorithms

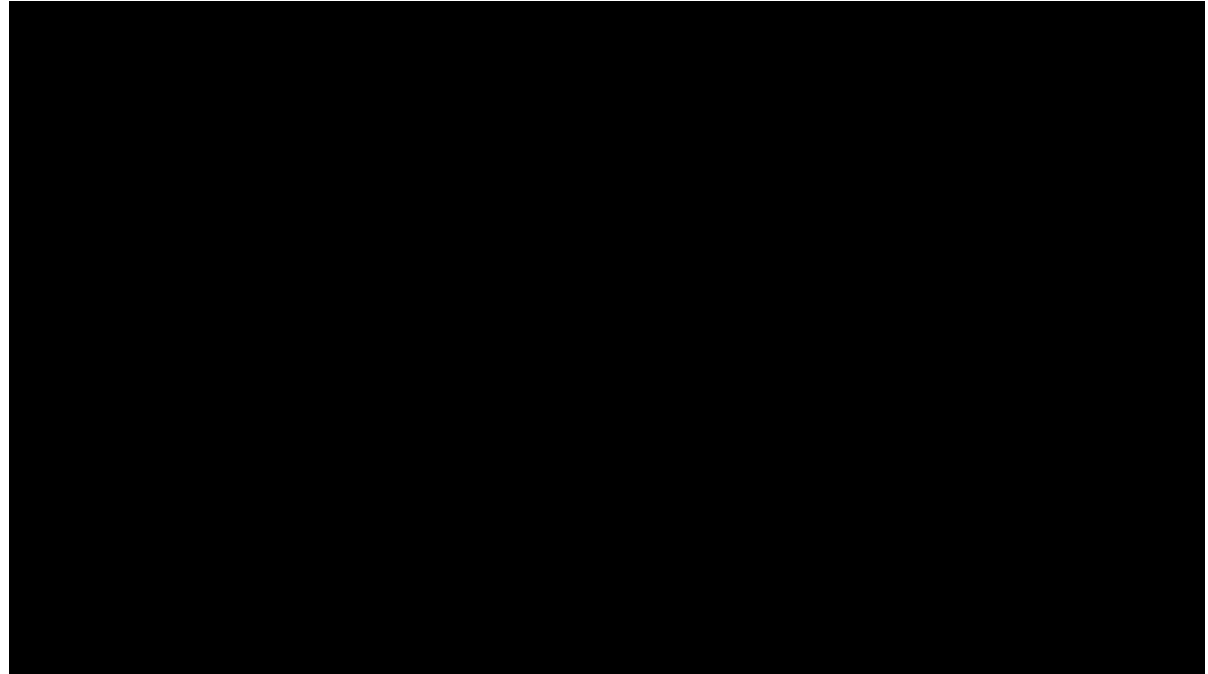
- Quicker operations (for example quicker sphere-sphere intersection tests)
- Cut down the amount of data to be processed:

SPATIAL DATA STRUCTURES



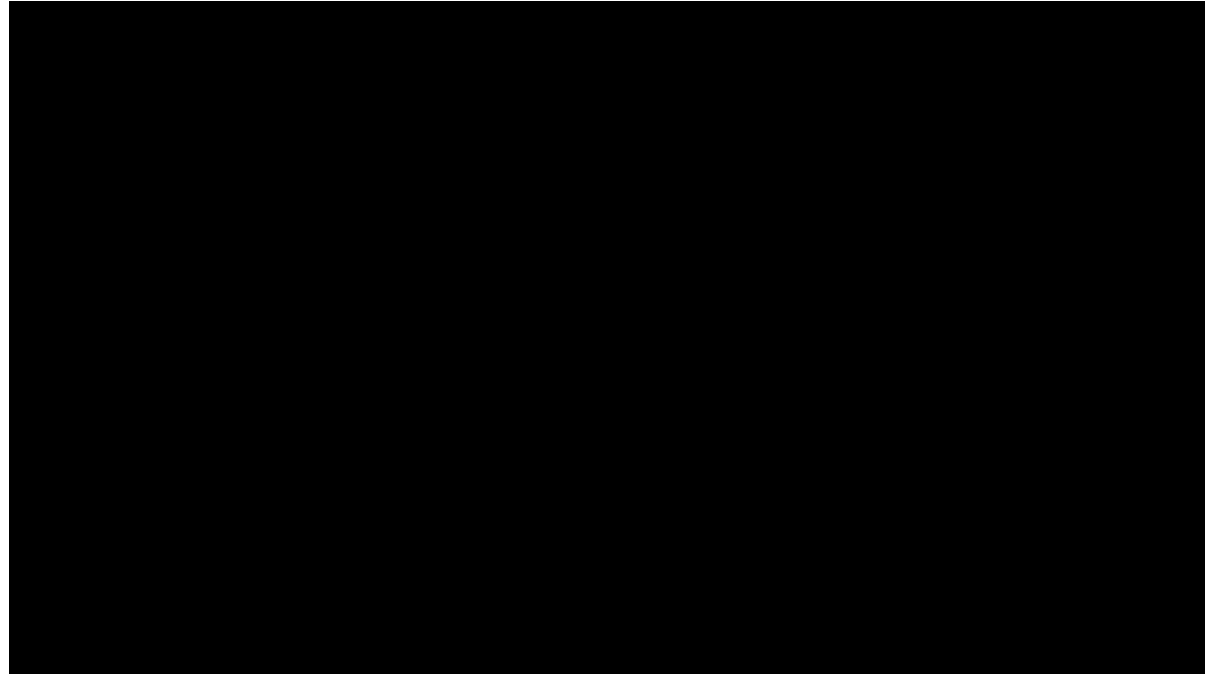
Uses

- ▶ Games
 - Graphics



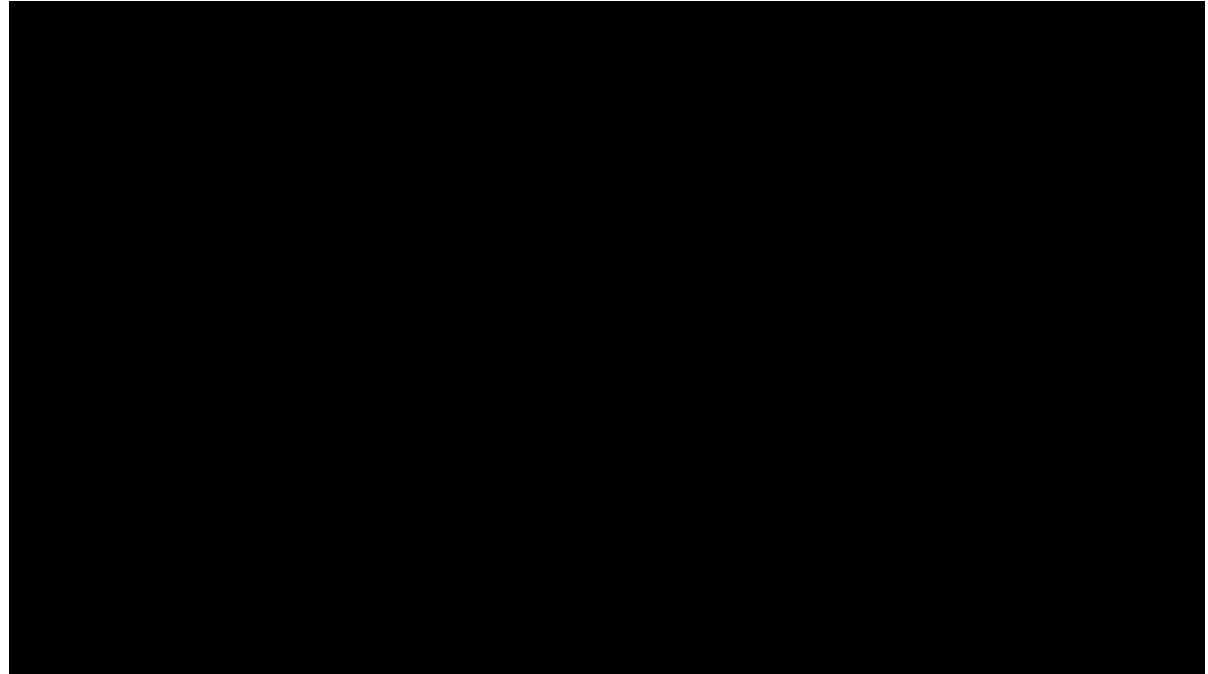
Uses

- ▶ Games
 - Graphics



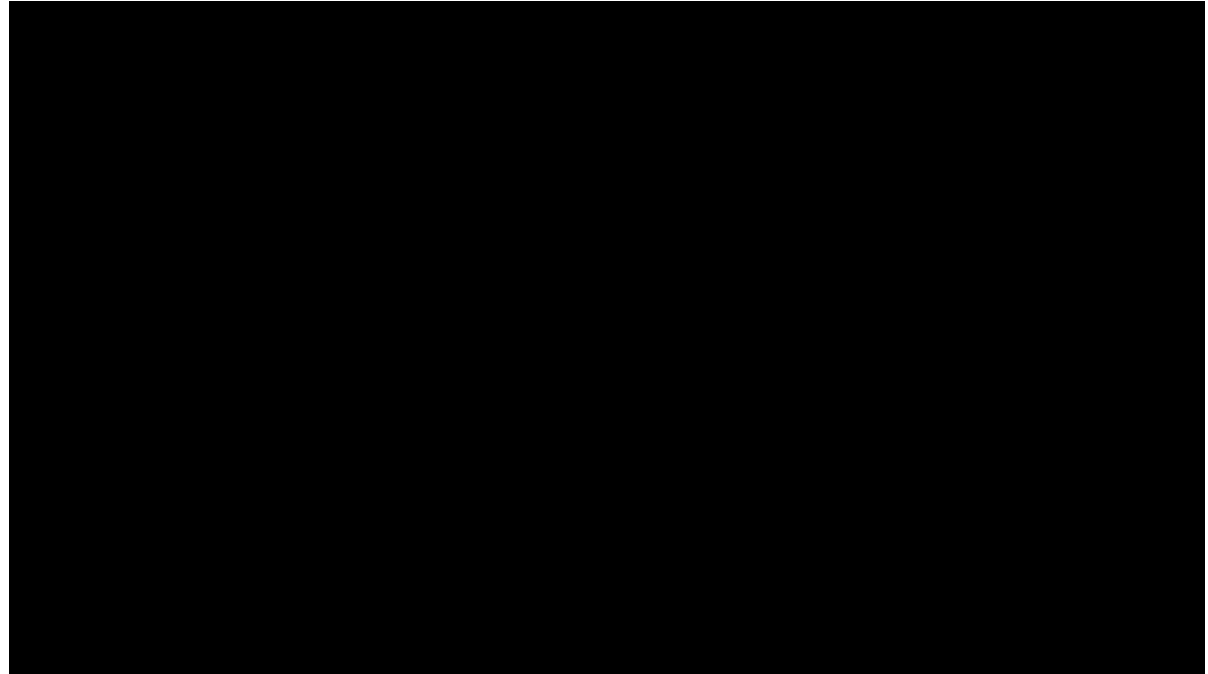
Uses

- ▶ Simulations
 - Physics



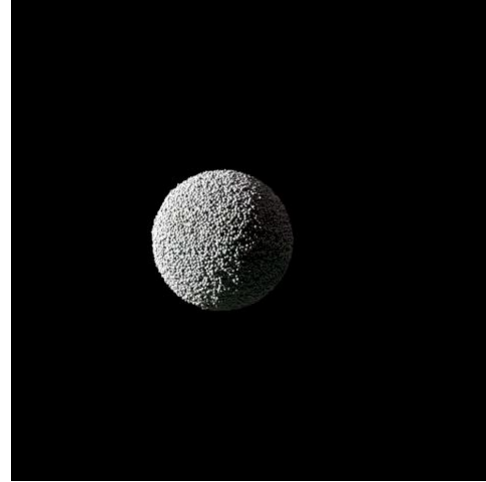
Uses

- ▶ Simulations
 - Physics



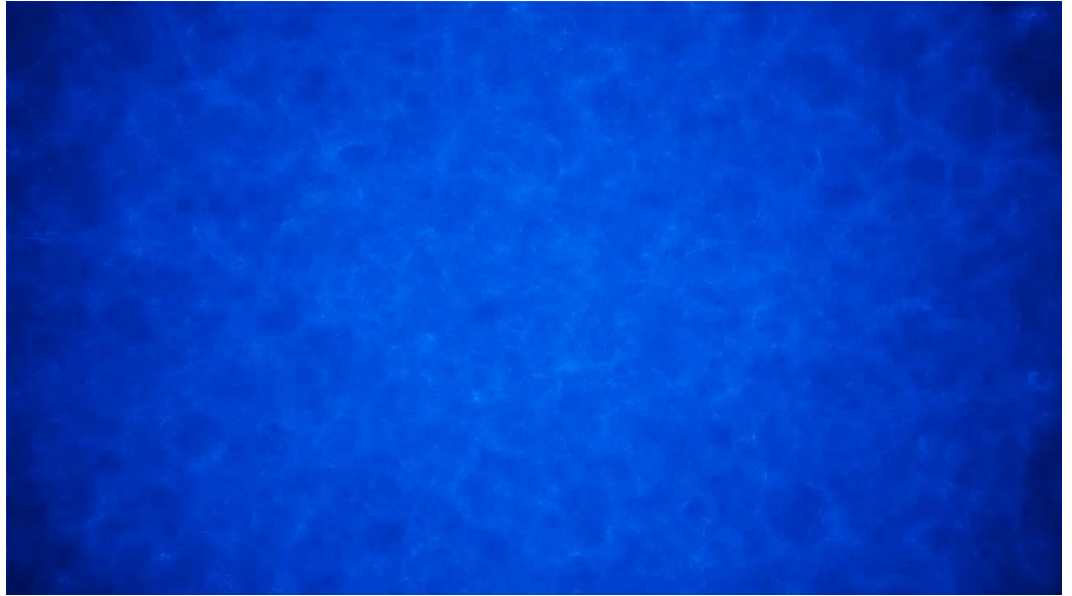
Uses

- ▶ Simulations
 - Astronomical



Uses

- ▶ Simulations
 - Astronomical



Uses

- ▶ Simulations
 - Fluid dynamics



Uses

► General algorithms

- Searching for related items in high dimensional subspaces
 - Image Processing
 - Machine Learning
 - Big Data



Pre-requisites

- ▶ Primitives in a d -dimensional vector space:
 - Points
 - Triangles / Quads
 - Implicit functions
- ▶ Will stick to 2D here (easier to draw!)



Useful Operations

- ▶ Set of primitives P
- ▶ Max, min, average
- ▶ Very similar process used to calculate each quantity: Max:

$\text{max} = -\infty$

For all p in P

$\text{max} = p > \text{max} ? p : \text{max}$



Useful Operations (2)

Min:

$\text{min} = \infty$

For all p in P

$\text{min} = p < \text{min} ? p : \text{min}$

Average:

$\text{average} = 0$

For all p in P

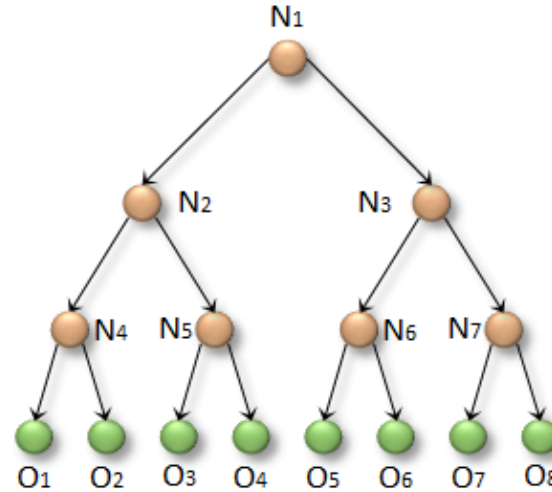
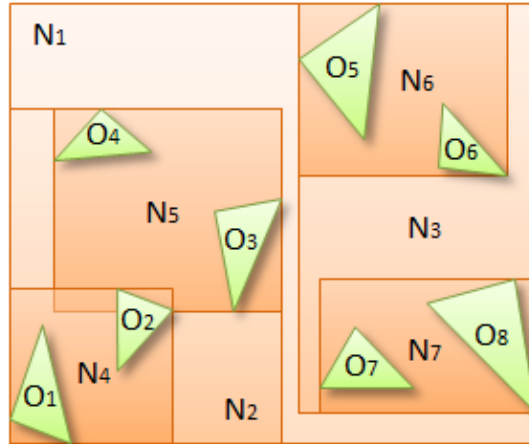
$\text{average} = \text{average} + p$

$\text{average} = \text{average} / |P|$



Idea

- ▶ Partition space into regions in which primitives can be located efficiently



Idea

- ▶ Many structures which allow this:
 - Grids
 - Quad/Oct-Tree
 - BVH
 - KD-Tree
 - BSP / BIH / QBVH...

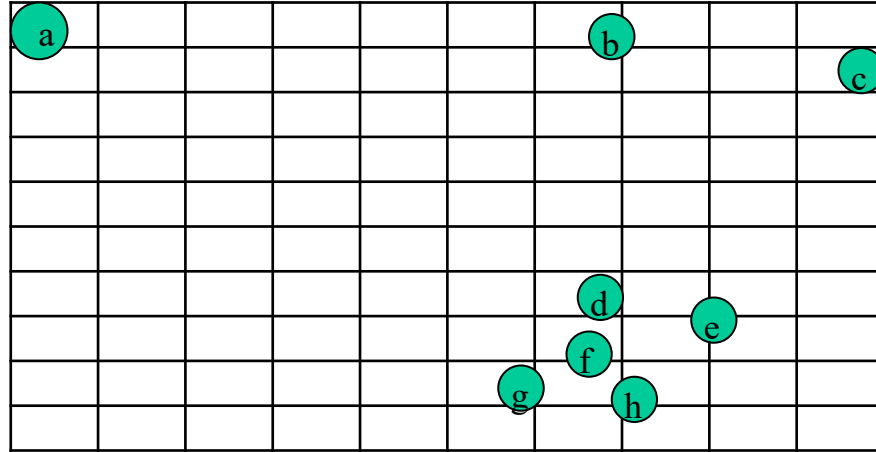


Operations

- ▶ Construct
- ▶ Search / Traverse
- ▶ (Insert, Delete and so on)



Grids



Grids

► Construction

- Find which cells (voxels if more than 2D) encompass each primitive
- Add a reference to the primitive in the relevant cell (linked list / vector etc)
- Linear in memory



Grids



Search

- Lookup based on coordinates
 - $\text{norm_coord} = (\text{coord} - \text{grid_min}) / (\text{grid_max} - \text{grid_min})$
 - $\text{index} = \text{floor}(\text{norm_coord} \times \text{gridsize})$



Grids

▶ Traversal

- Digital Differential Analyser for rays
 - Find step size based on ray direction
 - Step from cell-to-cell until intersection found or ray leaves grid



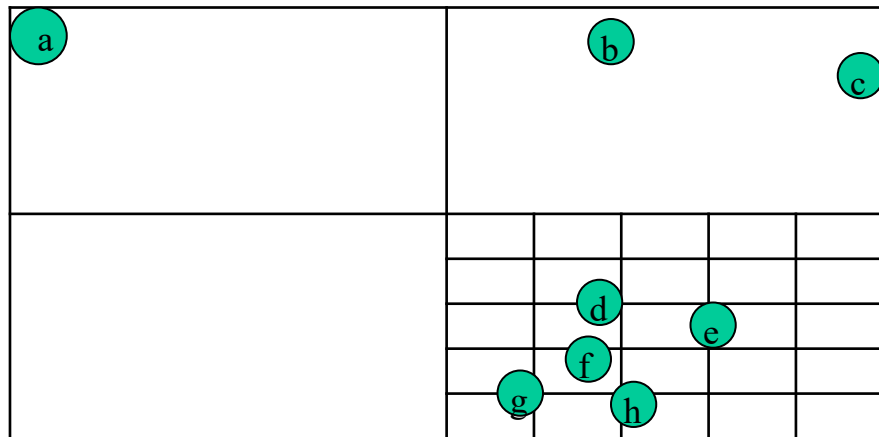
Grids

► Problems

- Memory intensive, especially in high dimensions (100, 10000, 1000000...)
- Wasted space / too many references in few voxels (teapot in a stadium)



Hierarchical Grids



Hierarchical Grids

- ▶ Subdivide grid when too many references in a voxel
- ▶ Partially alleviates the problem



Grids

- ▶ Simple spatial data structure
- ▶ Very simple and quick to build
- ▶ Copes very well with dynamic data
- ▶ Ray traversal less efficient than other structures
- ▶ Explosion of memory in higher dimensions

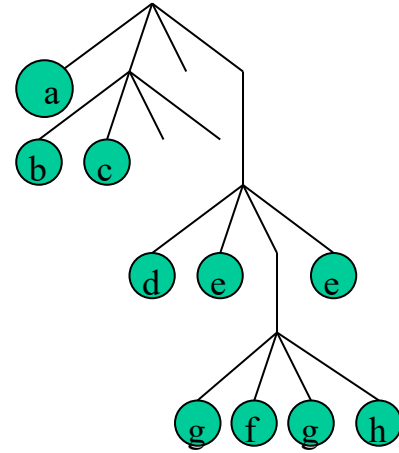
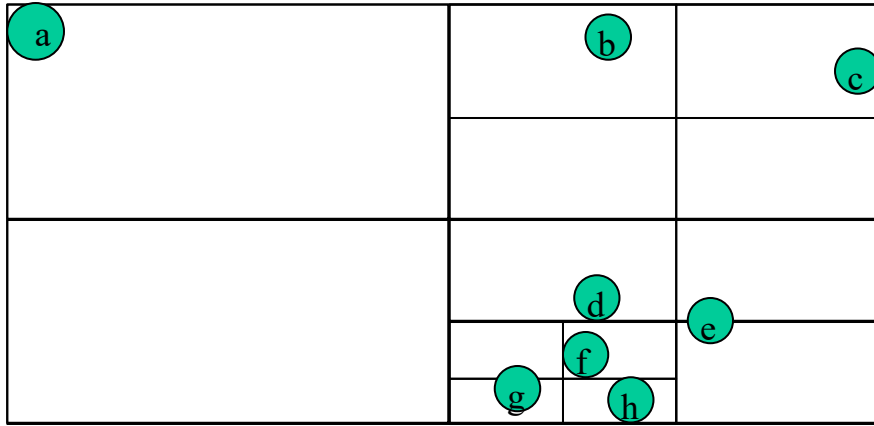


Quad-Tree

- ▶ Split 2D-Space more effectively
- ▶ Create a tree with four children
 - Construction recursively subdivides space
 - Can be stored as an array



Quad-Tree



Quad-Tree

▶ Traversal is recursive:

- Check if node is leaf
 - Process primitives
- Else
 - Inspect four children



Moving up dimensions

- ▶ Quad-trees work in 2D
- ▶ What about 3D?
- ▶ Logical next step is oct-tree



Oct-Tree

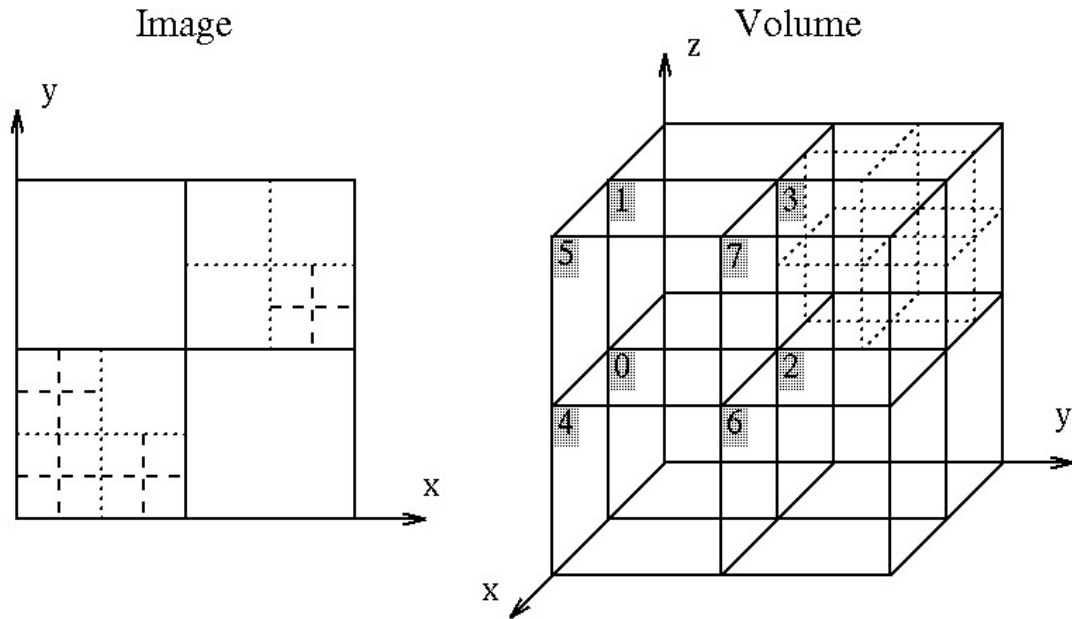


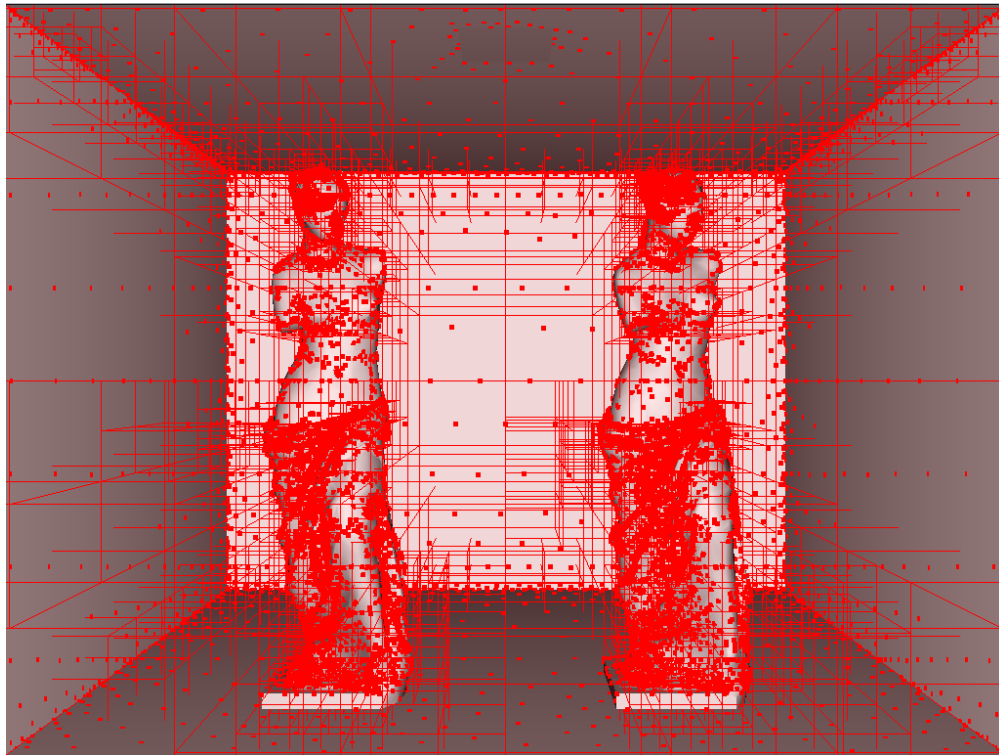
Image courtesy of <http://www-graphics.stanford.edu/~tolis/courses/cs161-97-summer/project/description.html>

Oct-Tree

- ▶ Eight children per node
- ▶ Similar construction and traversal as Quad-Tree
- ▶ Insertion / deletion are interesting cases:
 - Split octants if full
 - Merge if empty



Oct-Tree



Oct-Tree

- ▶ Good build times
- ▶ Good traversal times
- ▶ Can lead to wasted space (but not much)

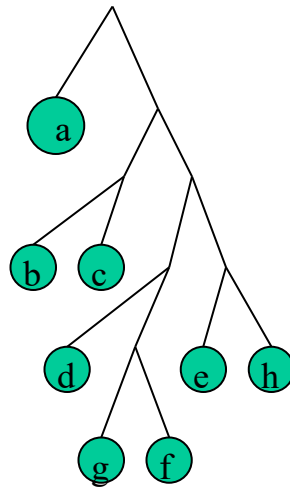
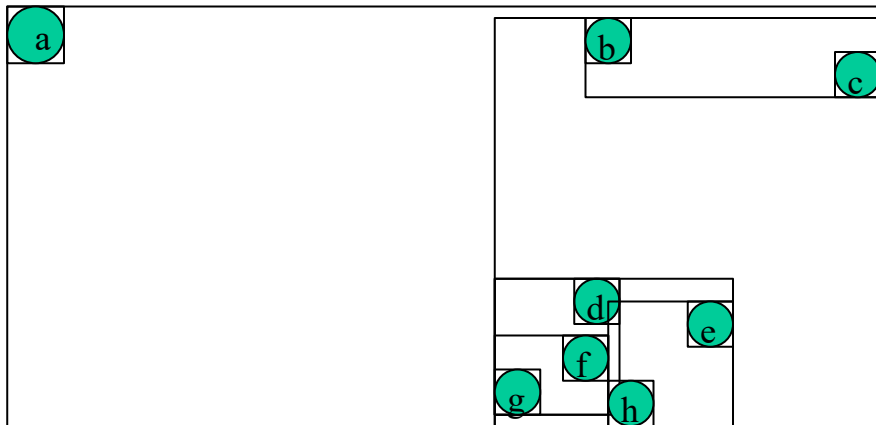


Bounding Volume Hierarchy

- ▶ Partition space into a tree of volumes
 - Primitives in leaves
 - Volumes higher in the tree (usually AABB)
 - Volumes can overlap



Bounding Volume Hierarchy



BVH Levels

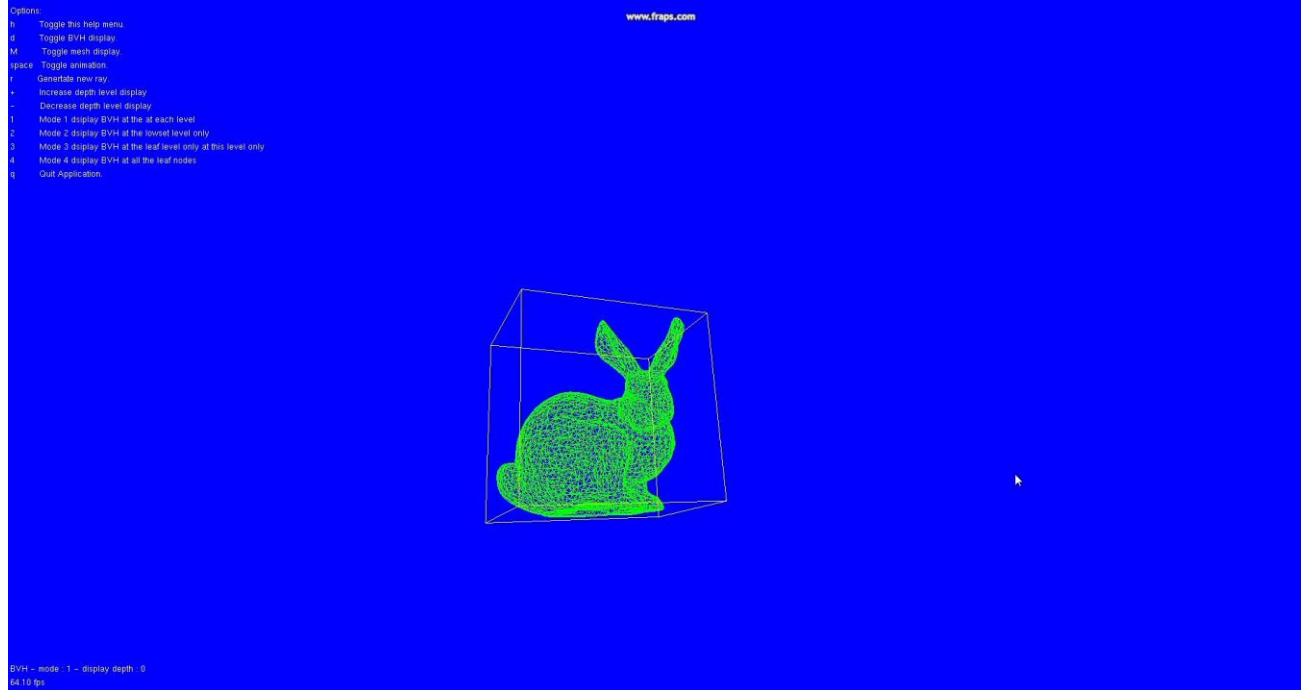


Image / Video courtesy of <http://www.mahmoudzidan.com/msclog/>

Bounding Volume Hierarchy

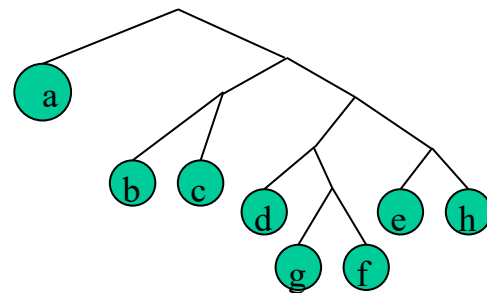
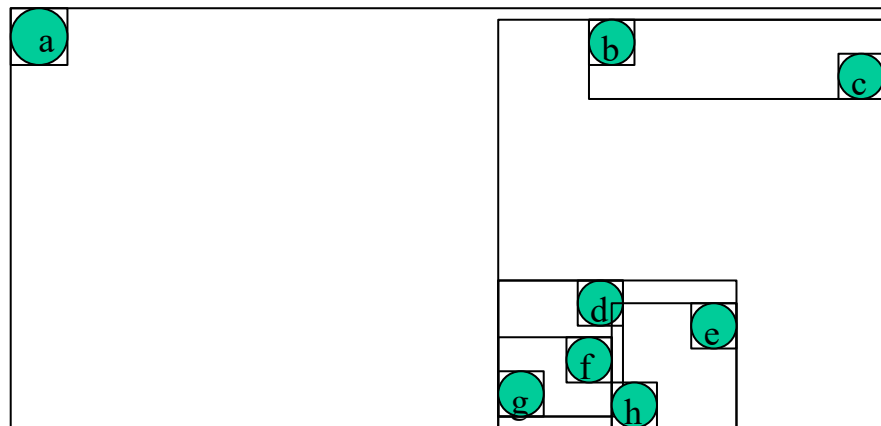
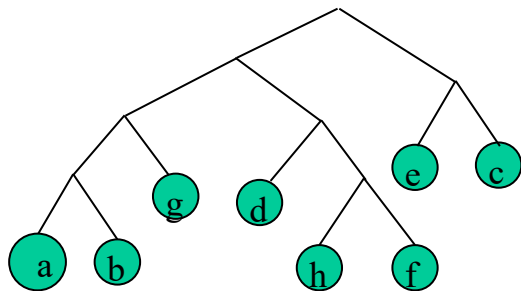
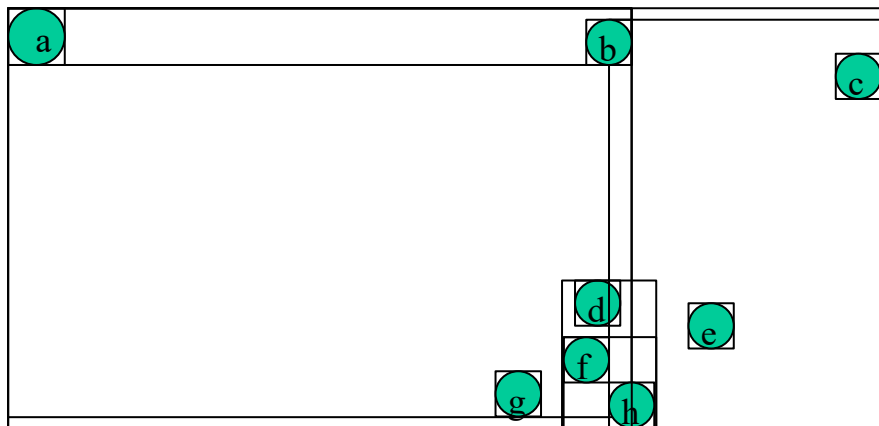
- ▶ Partition space into a tree of volumes
 - Good splits important higher in the tree
 - Nodes should be spatially local further down
 - Is a balanced tree always good?



Bounding Volume Hierarchy

- ▶ Construction
 - Top-down or bottom-up
- ▶ Can split at median values
 - Problems?
- ▶ Surface Area Heuristic
 - Estimate the cost of creating a sub-tree vs cost of computing with the data if it were a leaf

Bounding Volume Hierarchy



Surface Area Heuristic

- ▶ Based on costs (e.g. CPU cycles)
 - Cost for intersecting a primitive C_{isect}
 - Cost for intersecting the bounds C_{bounds}
- ▶ Cost for creating a leaf: $C_{\text{leaf}} = N C_{\text{isect}}$
 - Where N is the number of primitives

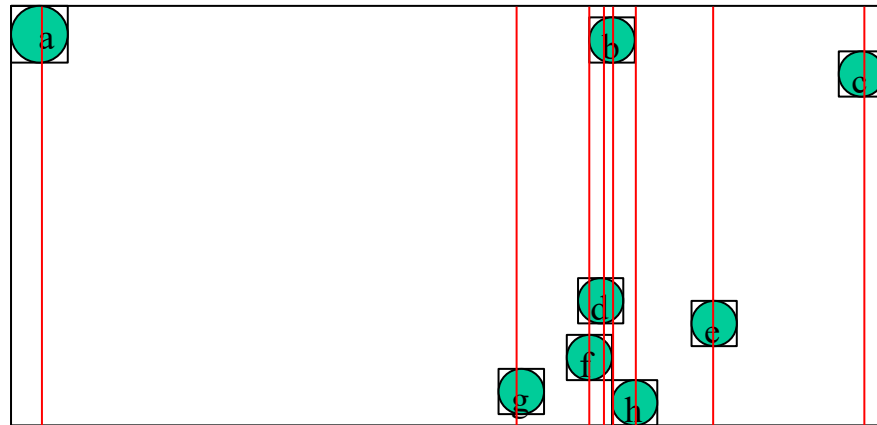
Surface Area Heuristic

- ▶ Check multiple ways to split the primitives
 - Loop of each axis and calculate many split candidates corresponding to each primitive in each axis
 - Binned SAH
 - Compute a fixed number of candidates



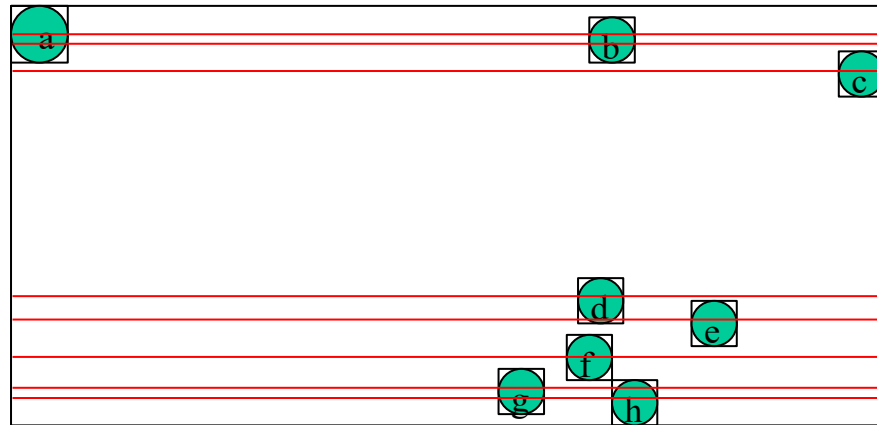
Surface Area Heuristic

- ▶ Check each primitive
 - Sort along each axis and check bounds



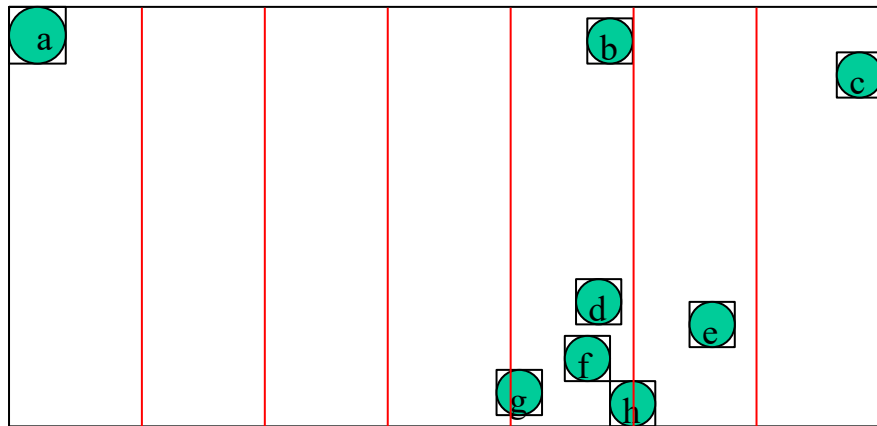
Surface Area Heuristic

- ▶ Check each primitive
 - Sort along each axis and check bounds



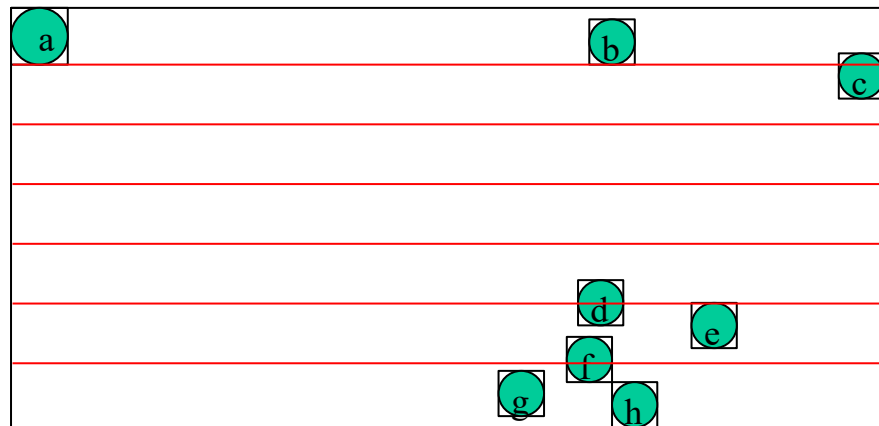
Surface Area Heuristic

- ▶ Binned SAH
 - Smaller number of candidates



Surface Area Heuristic

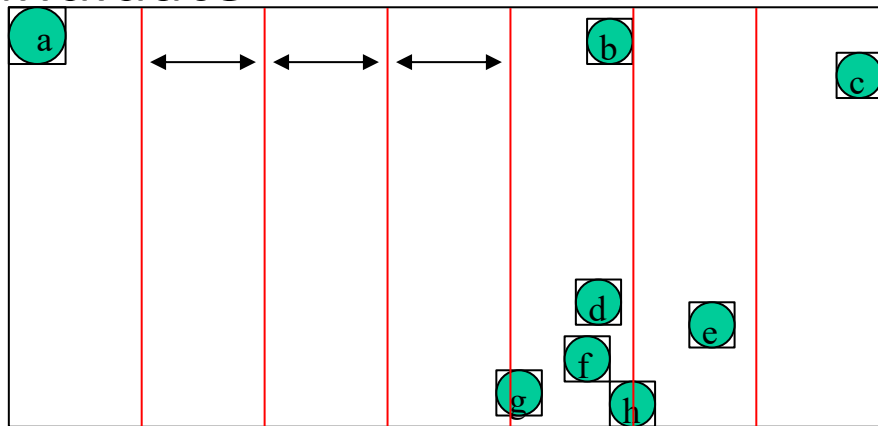
- ▶ Binned SAH
 - Faster to build



Surface Area Heuristic

► Binned SAH

- Calculate step size for each axis
- Step through each candidate



Surface Area Heuristic

- ▶ Cost to create a subtree given a split:

$$C_{\text{split}} = C_{\text{bounds}} + \frac{A(L)}{A(P)} N_L C_{\text{isect}} + \frac{A(R)}{A(P)} N_R C_{\text{isect}},$$

- ▶ Where

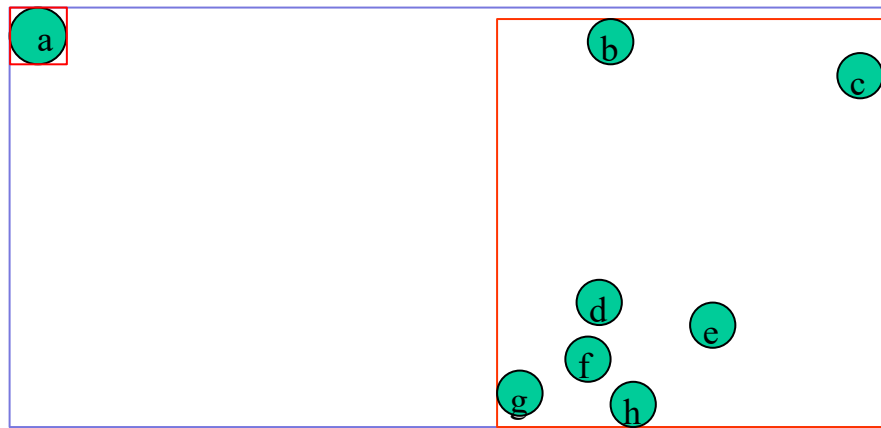
- Area of parent bounds $A(P)$
- Area of bounds of left $A(L)$ and right $A(R)$ nodes
- Number of primitives in left N_L and right N_R nodes



Surface Area Heuristic

► Motivation:

- Probability of a ray hitting a given bounding volume is proportional to its surface area relative to the parent bounding volume



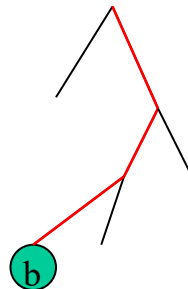
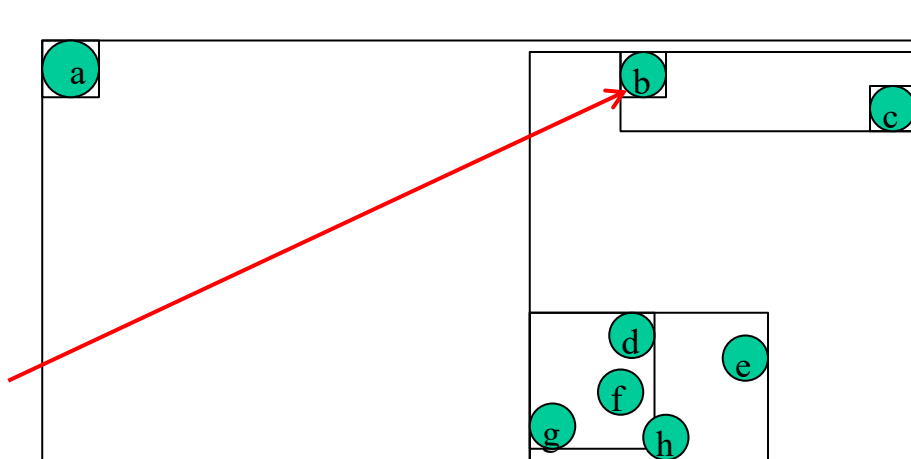
Traversal

► Recursive process

- Start at root
 - If Ray-AABB intersection
 - If leaf
 - » Process primitives in leaf
 - Else
 - » Check Left Child
 - » Check Right Child



Traversal



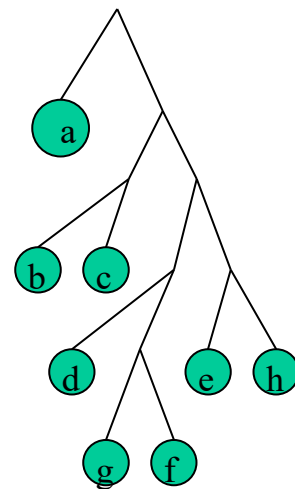
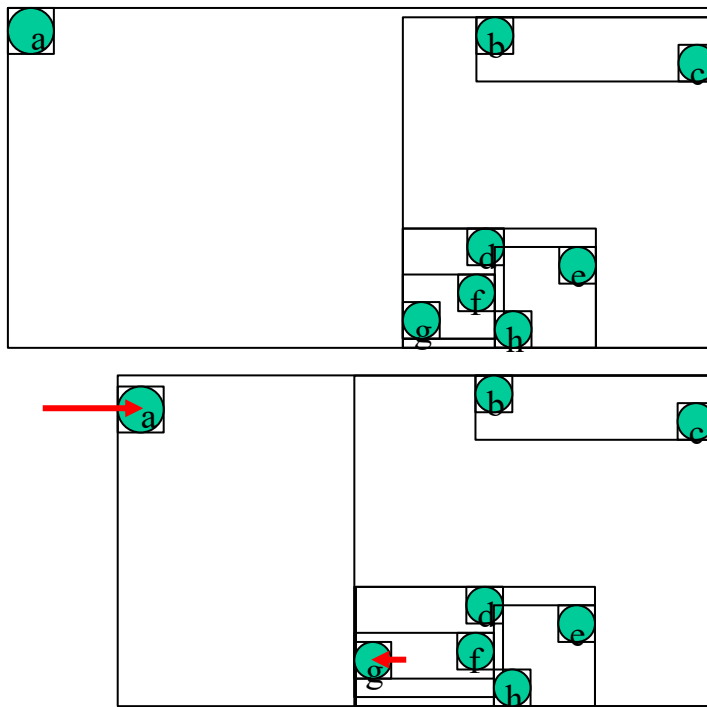
Bounding Volume Hierarchy

- ▶ Use overlapping volumes to partition space
- ▶ Can be quick to build
- ▶ Quick to traverse
- ▶ Volumes can be inserted / resized to cope with dynamic data



Bounding Volume Hierarchy

► Refitting

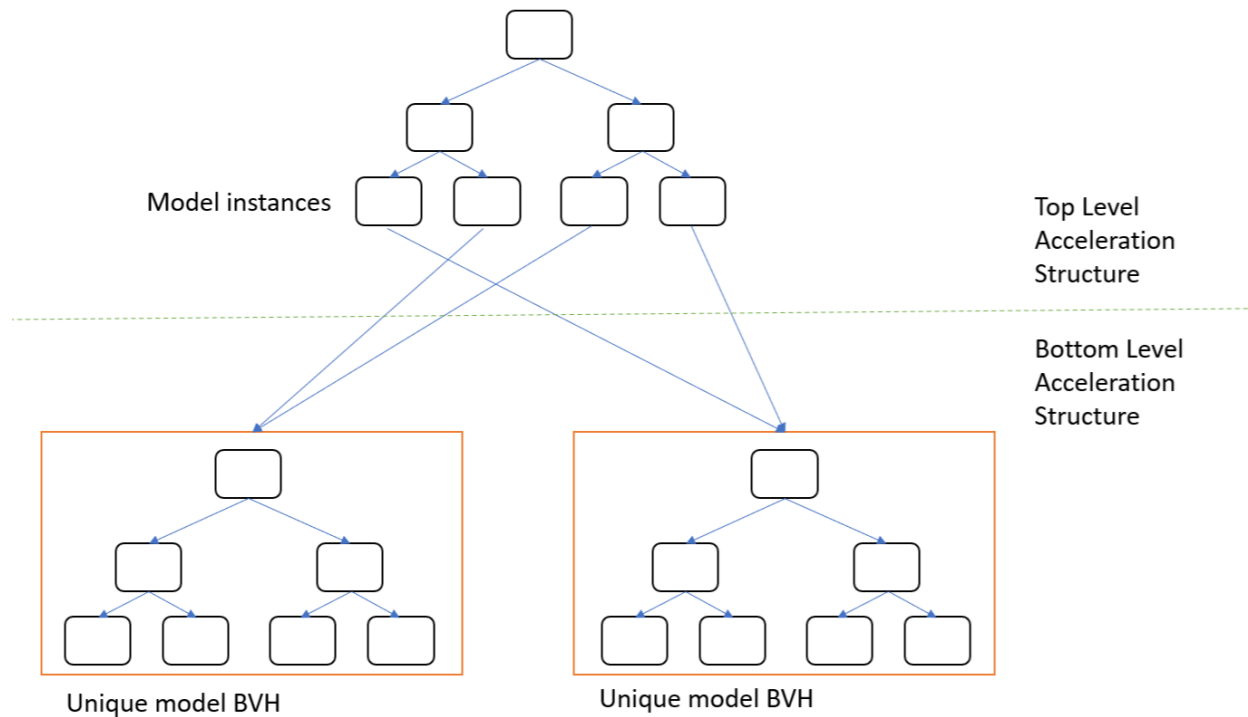


Two Level BVH

- ▶ Two BVHs
 - Bottom Level BVH stores BVH for each individual mesh (BLAS)
 - Top Level BVH stores BVH of instances of meshes (TLAS)
- ▶ Used on GPUs (and many CPU ray tracers)

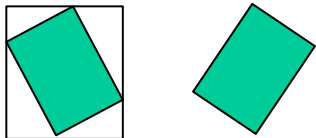


Two Level BVH



Two Level BVH

- ▶ TLAS stores reference to an instance of a BLAS
 - Also stores world matrix W_{inst}
 - Transform ray by W_{inst} when intersecting BLAS



KD-Tree

- ▶ Partitions space via axis aligned hyperplanes
- ▶ Slower build times often, but very fast traversal (used in light transport simulation for fast access to 3D, or sometimes 6D data)

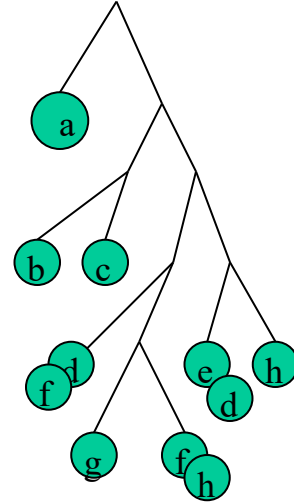
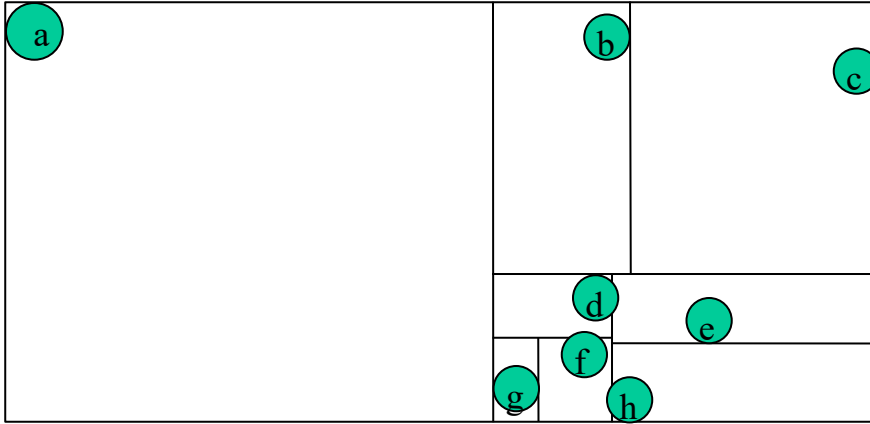
KD-Tree

► Construction

- Similar to BVHs *except* primitives cannot overlap volumes
- Two strategies for this:
 - Easy, add a reference to both sides of a split
 - Harder, split the primitive along the splitting axis
- Generally top-down



KD-Tree



KD-Tree

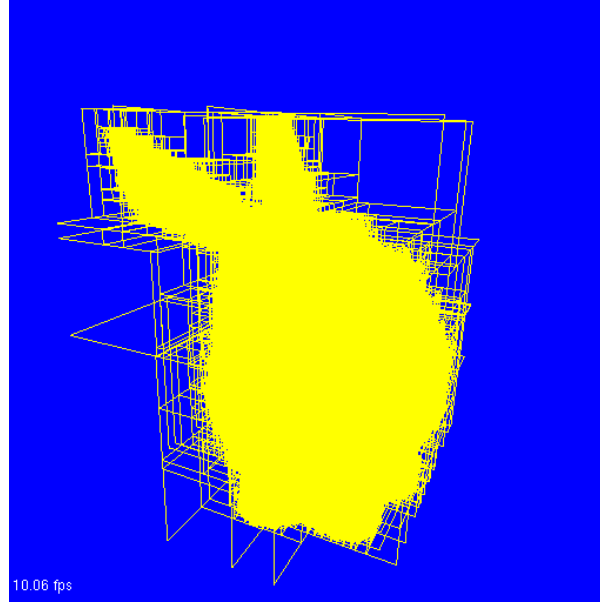


Image courtesy of <http://www.mahmoudzidan.com/msclog/>

KD-Tree

▶ Traversal

- Two interesting operations
 - Nearest neighbour
 - Ray traversal



KD-Tree

- ▶ Nearest Neighbour (Node node, Point x, search_radius)

If node == leaf

 If $|p - x| < \text{search_radius}$

 set p as current closest to x

 search_radius = $|p - x|$

Else

 If (inside(node.l, x, search_radius))

 NearestNeighbour(node.l, x)

 If (inside(node.r, x, search_radius))

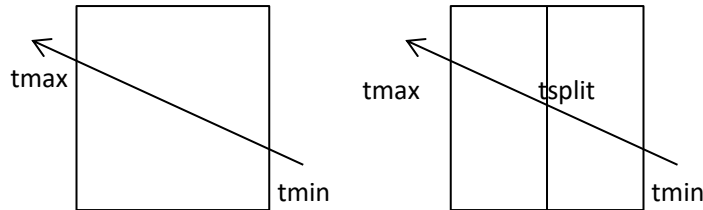
 NearestNeighbour(node.r, x)

- ▶ Can generalise to K-Nearest Neighbour

KD-Tree

► Ray Intersection:

- Uses min and max bound for the ray
- Updated as ray intersects with hyper-plane
- Processes children in depth order



Other types of tree

▶ BSP

- Binary tree of arbitrary aligned split planes which split the data set into two each time
- Split planes can be calculated with PCA
- Can lead to numerical instabilities in traversal
- Slower build times, and slower traversal due to non-aligned split planes



Other types of tree (2)

▶ BIH

- Conceptually a combination of BVH and KD-Tree
- Two split planes per node
- Fast, low memory, quick to build, more numerical stability



Other types of tree (3)

► QBVH

- BVH with four children per node
- Create from BVH
 - Merge every $N + 1$ 'th level
 - Delete every N 'th level



Other types of tree (3)

▶ QBVH

- Can be faster
 - Currently considered the fastest on CPU (SIMD efficiency): Intel Embree
- Can extend to more children



Spatial Hashing



Idea

- Create a hash for ND space
- Store data in hash table



Spatial Hashing

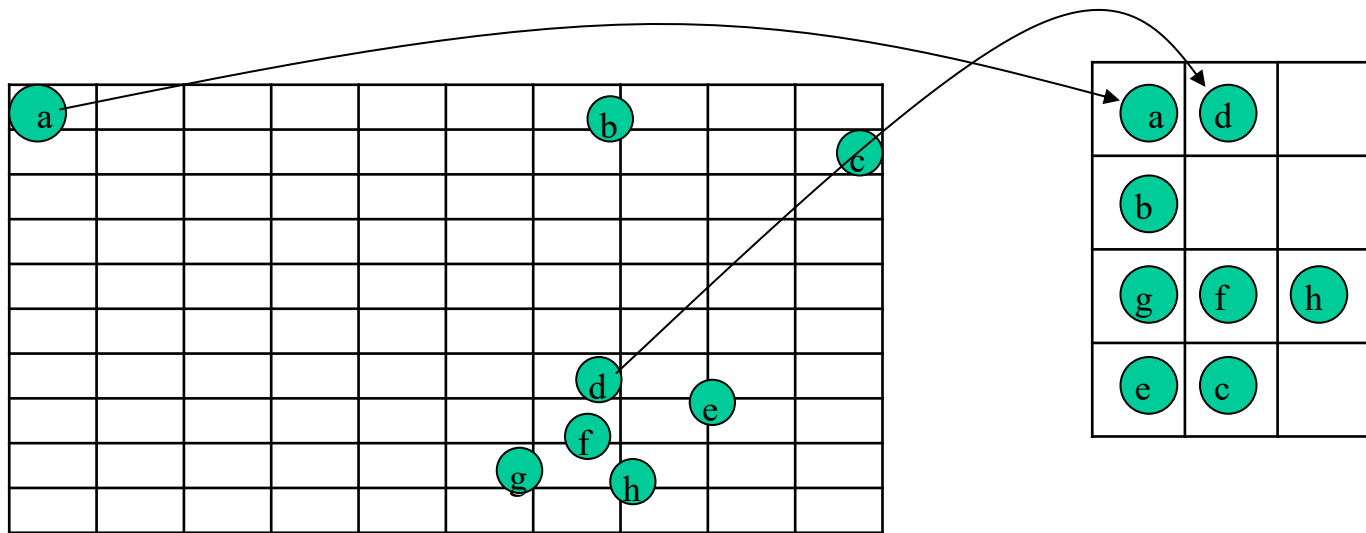
► Use coordinates to generate hash:

– e.g. for 3D coordinates:

```
unsigned int spatialHash(float x, float y, float z)
{
    int xi = (int)floorf(x);
    int yi = (int)floorf(y);
    int zi = (int)floorf(z);
    return ((73856093 * xi) ^ (19349663 * yi) ^ (83492791u * zi)) % size;
}
```

– Prime numbers help to distribute hash values uniformly, which reduces collisions

Spatial Hashing



Spatial Hashing

- ▶ Fast to build
- ▶ Very efficient storage
- ▶ Fast to find primitives
- ▶ Inherits limitations of hash tables



Implementation

- ▶ Assignment: Build a BVH over the triangles in your system
- ▶ Add SAH building
- ▶ Add traversal code
- ▶ Link into Scene Class



Implementation

▶ BVHNode Class

```
class BVHNode
{
public:
    AABB bounds;
    BVHNode* r;
    BVHNode* l;
```


Implementation

► Primitive options

- Store offset and size in a reordered list of triangles

```
unsigned int offset;  
unsigned char used;
```

- Store primitives / pointers to primitives directly

```
Triangle *triangles;  
unsigned char num;
```

- Many other options



Implementation

► Building

- Add to BVHNode class
- e.g.

```
void build(std::vector<Triangle>& inputTriangles, std::vector<Triangle>& outputTriangles)
```

► Add code to calculate bounds, SAH split planes, and building sub trees



Implementation

► Building

– Call from `void build()` in Scene class

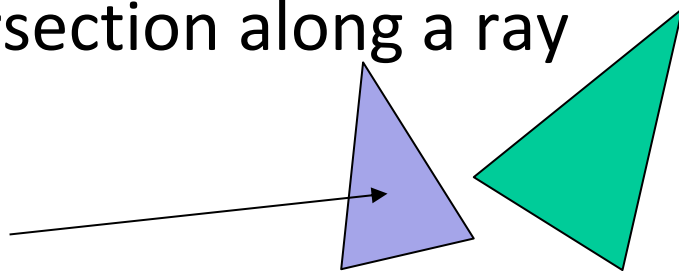
– e.g. `void build()`
`{`

```
// Add BVH building code here
std::vector<Triangle> inputTriangles;
for (int i = 0; i < triangles.size(); i++)
{
    inputTriangles.push_back(triangles[i]);
}
triangles.clear();
bvh = new BVHNode();
bvh->build(inputTriangles, triangles);
```

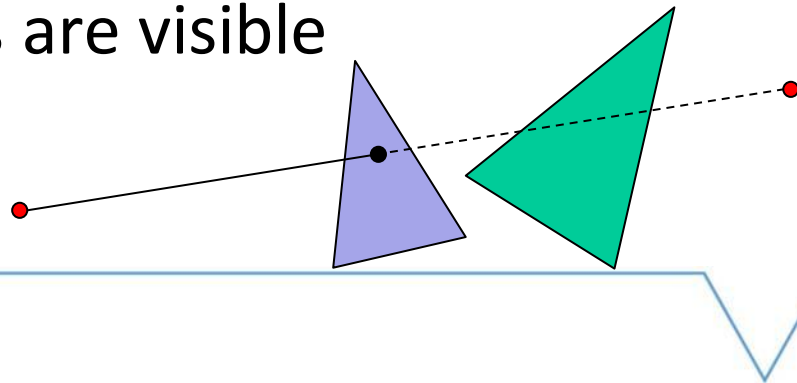
Implementation

► Traversal Code

- Find closest intersection along a ray



- Find if two points are visible



Implementation

► Find closest intersection

```
void traverse(const Ray& ray, const std::vector<Triangle>& triangles, IntersectionData& intersection)
```

- Check bounds
 - If intersects:
 - » If not leaf, traverse children nodes
 - » Else ray-triangle for all triangles in node
- Global list of triangles

Implementation

► Find closest intersection

- Call with initialized `IntersectionData`

```
IntersectionData traverse(const Ray& ray, const std::vector<Triangle>& triangles) {  
    IntersectionData intersection;  
    intersection.t = FLT_MAX;  
    traverse(ray, triangles, intersection);  
    return intersection;  
}
```

- Link to Scene class

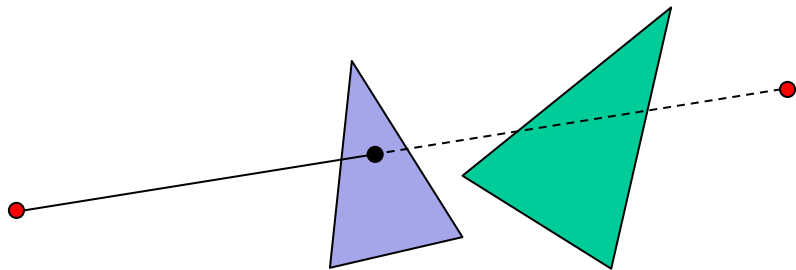
```
IntersectionData traverse(const Ray& ray)  
{  
    return bvh->traverse(ray, triangles);  
}
```



Implementation

► Find visible

- Similar to traverse, but return false as soon as a primitive is intersected
 - Why?



Implementation

► Find visible

- Similar to traverse, but return false as soon as a primitive is intersected

```
bool traverseVisible(const Ray& ray, const std::vector<Triangle>& triangles, const float maxT)
```

- Link to Scene class

```
bool visible(const Vec3& p1, const Vec3& p2) {  
    Ray ray;  
    Vec3 dir = p2 - p1;  
    float maxT = dir.length() - (2.0f * EPSILON);  
    dir = dir.normalize();  
    ray.init(p1 + (dir * EPSILON), dir);  
    return bvh->traverseVisible(ray, triangles, maxT);  
}
```




Questions?

Advanced Computer Graphics

Spatial Data Structures

WARWICK

Dr. Tom Bashford-Rogers